SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Conflict Free Exam Timetable Scheduling System Using Graph Coloring and Backtracking Algorithms

A PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA0614-Design and Analysis of Algorithm

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Computer Science Engineering

Submitted by

K. Jaya Venkata Sai (192525182)

G. Rama Krishna (192565028)

Athavan K (192524036)

Manoj Kumar K (192521041)

Under the supervision of

Dr. A. Sairam

August 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, K. Jaya Venkata Sai, G. Rama Krishna, Athavan K, and Manoj Kumar K, of the B.Tech Computer Science Engineering, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled 'Conflict Free Exam Timetable Scheduling System Using Graph Coloring and Backtracking Algorithms' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: CHENNAI

Date: _____

Signature of the Students with Names

K. Jaya Venkata Sai

G. Rama Krishna

Athavan K

Manoj Kumar K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Conflict Free Exam Timetable Scheduling System Using Graph Coloring and Backtracking Algorithms” has been carried out by K. Jaya Venkata Sai, G. Rama Krishna, Athavan K, and Manoj Kumar K under the supervision of Dr. A. Sairam and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech (Computer Science Engineering) program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr A Sairam

Professor

Dept. of Computer Science Engineering

SIMATS

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department for continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide for creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members, and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work.

Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name
K. Jaya Venkata Sai
G. Rama Krishna
Athavan K
Manoj Kumar K

ABSTRACT

Examination timetabling is a constrained scheduling problem in which two courses sharing at least one student cannot be placed in the same time slot. This report models the problem as proper graph colouring: courses are vertices, shared-student conflicts are edges, and available examination periods are colours. The supplied Annexure A graph contains six courses and eight conflict edges, including a triangle among C1, C2 and C3 that establishes a lower bound of three colours.

The solution is organised into four modules. The Graph Model & Data Layer stores the hardcoded graph and produces deterministic randomized graphs with $n \geq 15$ for scalability experiments. Part A implements plain recursive backtracking, colouring vertices in a fixed order and trying colours from 1 through k. Part B adds two pruning mechanisms: Most-Constrained-Vertex selection and Forward Checking. Part C checks the chromatic number, collects trace and performance metrics, generates charts, and derives the $O(k^n)$ worst-case bound.

For the Annexure A graph, the chromatic number is 3: two colours are infeasible because of the triangle, while three colours produce a valid timetable. On the seeded scalability set n = 15, 20, 25, 30, 35, 40 and 45, the pruning-enhanced solver substantially reduces colour attempts and backtracking events. The practical conclusion is not that pruning changes the worst-case class, but that it avoids large portions of the search tree on structured conflict graphs, making it the preferred final solver when traceability and moderate scalability are required.

Keywords: graph colouring, examination scheduling, backtracking, forward checking, constraint satisfaction, chromatic number, algorithm analysis.

CONTENTS AND LISTS

1. PROBLEM STATEMENT AND FORMULATION

1.1 Problem context

A university must schedule final examinations for a set of courses. A student enrolled in two courses creates a conflict if both examinations are assigned to the same time slot. Manual scheduling becomes difficult as the number of courses, student overlaps, rooms and available periods increases. The assignment isolates the central algorithmic challenge: avoid simultaneous exams for conflicting courses while using as few time slots as possible.

The key modelling move is to represent each course as a vertex in an undirected graph. An edge joins two vertices when the corresponding courses share at least one student. A colour represents an examination time slot. A proper colouring assigns one colour to every vertex and ensures that adjacent vertices have different colours.

1.2 Mathematical formulation

Let G = (V, E) be the conflict graph, where V = {C1, C2, …, Cn}. Let K = {1, 2, …, k} be the available colour set. A colouring is a function f: V → K. The feasibility constraint is:

For every (u, v) ∈ E: $f(u) \neq f(v)$

The decision version asks whether G is k-colourable. The optimization version asks for the smallest k for which a feasible colouring exists. That smallest value is the chromatic number $\chi(G)$. The implementation first solves the decision problem for a fixed k and then repeats it from k = 1 upward for the chromatic-number check.

1.3 Annexure A interpretation

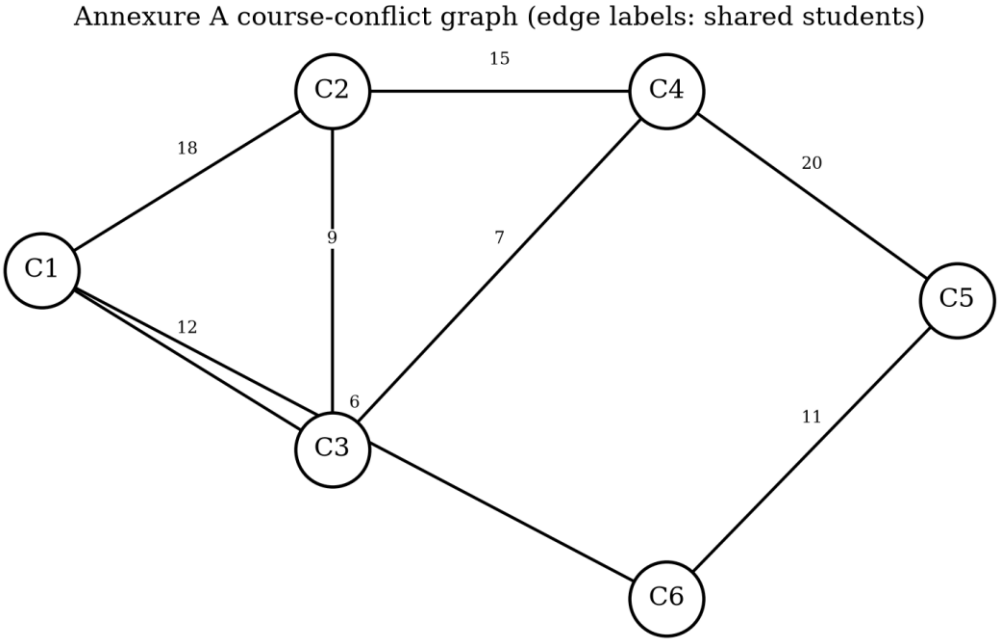

Figure 1. Annexure A graph used for the hand-traced walkthrough. Numbers beside edges are shared-student counts.

Edge	Shared students	Scheduling meaning
C1 – C2	18	C1 and C2 must use different slots.
C1 – C3	12	C1 and C3 must use different slots.
C2 – C3	9	C2 and C3 must use different slots.
C2 – C4	15	C2 and C4 must use different slots.
C3 – C4	7	C3 and C4 must use different slots.
C4 – C5	20	C4 and C5 must use different slots.
C5 – C6	11	C5 and C6 must use different slots.
C1 – C6	6	C1 and C6 must use different slots.

The triangle C1–C2–C3 is important: each pair is adjacent, so those three courses require three distinct colours. Therefore $\chi(G) \geq 3$. The constructed colouring C1=1, C2=2, C3=3, C4=1, C5=2, C6=3 proves $\chi(G) \leq 3$. Together, these bounds show $\chi(G) = 3$.

2. OBJECTIVES AND EXPECTED OUTCOMES

The report converts the assignment brief into observable objectives. Each objective is linked to an implementation output or a documented analysis rather than being treated as a purely descriptive statement.

Objective	Expected outcome	Evidence in this report
Model the scheduling problem	A conflict graph and adjacency representation are defined.	Sections 1 and 5; edge and adjacency tables.
Implement Part A	A fixed-order recursive solver tries colours in increasing order and backtracks on conflict.	Section 6; source appendix; trace tables.
Implement Part B	MCV ordering and Forward Checking reduce the effective search space.	Section 6; domain illustration; comparison tables.
Analyse Part C	Worst-case complexity, chromatic number and practical scaling are reported.	Sections 9 and 10; charts and derivation.
Validate correctness	Solutions satisfy every edge constraint; infeasible k values are rejected.	Section 8 test matrix and validation checklist.
Prepare GitHub deliverable	The four modules, README and reproducibility information are organised for upload.	Section 7 and Appendix D.
Reflect on learning	The student explains algorithmic decisions, trade-offs and improvements.	Section 13; replace placeholders before submission.

2.1 Success criteria

- Every returned colouring assigns a colour in the requested range 1..k.
- For every graph edge (u, v), the final colours are different.
- The solver returns failure for Annexure A with k = 2 and success with k = 3.
- The plain solver records assignment attempts, rejected candidates and backtracks.
- The enhanced solver records the same headline metrics and can be compared directly.
- The scalability experiment includes $n \geq 15$ and extends beyond n = 40.
- The final recommendation is supported by actual run data, not only by asymptotic theory.

2.2 Course outcomes and Bloom level

The work applies analysis rather than copying a textbook routine. It formulates a real scheduling situation (L4), designs and adapts recursive search (L4–L5), evaluates a heuristic against a baseline using measured evidence (L5), and connects the design to responsible engineering and SDG 4 (L4–L5).

Course/assessment focus	Applied evidence	Cognitive action
Problem formulation	Courses, conflicts, slots and constraints mapped to G.	Analyse
Algorithmic design	Plain solver and MCV + Forward Checking solver.	Design
Efficiency	Counts, runtime means and reduction percentages.	Evaluate
Decision	Trade-off table and final recommendation.	Justify
Reflection	Specific learning and improvement plan.	Reflect

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional requirements

ID	Requirement	Acceptance condition
FR1	Accept a graph with n vertices and undirected conflict edges.	Adjacency sets are built symmetrically.
FR2	Accept a fixed number k of available slots.	Values tried are exactly 1..k.
FR3	Return a feasible colouring or an explicit failure result.	No partial colouring is presented as success.
FR4	Implement fixed-order plain backtracking.	Vertex order is deterministic and traceable.
FR5	Implement MCV plus Forward Checking.	Variable selection and domain reduction are visible in the trace.
FR6	Check the chromatic number.	The first successful k is reported.
FR7	Generate randomized graphs for scale tests.	Seed and probability are recorded.
FR8	Export evidence for the report/GitHub.	Tables, charts, results.md and source files are available.

3.2 Non-functional and performance requirements

Category	Requirement	Design response
Correctness	Never place adjacent courses in one slot.	Conflict checks before every assignment; final validator.
Reproducibility	Experiments must be repeatable.	Fixed seeds, fixed n list and documented environment.
Traceability	Hand-trace steps must be inspectable.	Structured event records for try, assign, reject, undo and prune.
Scalability	Demonstrate n = 15 through at least 40.	Experiments use 15, 20, 25, 30, 35, 40, 45.
Maintainability	Separate concerns into modules.	Four-module source structure.
Readability	Report should be suitable for assessment.	Times New Roman, black text, clear captions and plain tables.

3.3 Constraints

Constraint	Impact
A conflict edge is binary	The shared-student number is retained as metadata; the core solver uses adjacency.
All exams are assumed equal duration	Only same-slot conflicts are modelled.
Rooms and invigilation are excluded	The graph captures student clashes, not the complete institutional problem.
No soft preferences	The objective is feasibility/minimum colours, not student convenience.
Backtracking is exponential in the worst case	Large dense graphs may require stronger CSP or optimization methods.
Random graphs are synthetic	They demonstrate scaling but do not replace institutional enrolment data.
No personal student data is processed	The experiment preserves privacy by using aggregate/synthetic conflicts.

3.4 Assumptions

- The graph is simple, undirected and contains no self-loops.
- Each course receives exactly one examination slot.
- The requested k is a positive integer.
- A valid colouring is sufficient for the assignment; room capacity and invigilator availability are future extensions.
- The randomized generator creates reproducible graphs using a seed and a planted three-colour structure.

4. APPLICATION OF COURSE KNOWLEDGE

4.1 Backtracking as systematic state-space search

Backtracking is a depth-first exploration of a decision tree. At depth i, the algorithm has assigned the first i variables in its chosen order. A branch is extended only when the partial assignment remains consistent. If every available value fails, the algorithm returns to the previous depth and tries a different value there.

The method is complete for finite graph-colouring instances: if a valid k-colouring exists, exhaustive exploration eventually reaches it; if no branch survives, the instance is infeasible for k. Completeness is obtained at the cost of potentially exponential work.

4.2 Constraint satisfaction perspective

CSP element	**Graph-colouring interpretation**
Variables	Courses C1, C2, …, Cn.
Domains	Available slots {1, 2, …, k}.
Binary constraints	Adjacent courses must have different values.
Assignment	A partial timetable.
Consistency test	No assigned neighbour has the candidate colour.
Propagation	Forward Checking removes a chosen colour from unassigned neighbours.

4.3 Recursion invariant

At the entry to every recursive call, all already-coloured vertices satisfy every edge constraint among them. The algorithm preserves this invariant because it assigns a candidate only after checking its coloured neighbours. When a recursive call fails, the assignment and all domain removals caused by that assignment are undone before the caller tries the next candidate.

4.4 Lower and upper bounds for χ(G)

A clique of size q gives $\chi(G) \geq q$ because every pair in the clique must receive a different colour. A valid colouring using k colours gives $\chi(G) \leq k$. For Annexure A, the triangle gives the lower bound 3 and the explicit colouring gives the upper bound 3. This is a short proof that avoids relying on the search output alone.

4.5 Why the heuristic is algorithmically meaningful

The enhanced solver does not change the legal solutions. MCV changes the order in which variables are exposed: a variable with the fewest remaining options is selected first, making a contradiction appear near the top of the search tree. Forward Checking performs local propagation after each assignment. Together, they reduce the number of descendants that need to be visited, although the worst-case upper bound remains exponential.

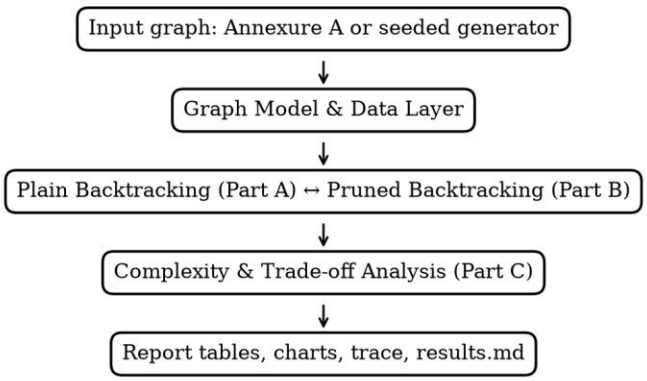

Figure 2. Four-module architecture and information flow.

5. DESIGN AND PROPOSED METHODOLOGY

5.1 Four-module design

Module	Inputs	Outputs	Assignment mapping
Graph Model & Data Layer	Annexure A edge list or n, p, seed	Graph object, adjacency sets, metadata	Data foundation / scalability
Plain Backtracking	Graph, k, fixed vertex order	Colouring, success flag, trace, metrics	Part A
Pruning-Enhanced Backtracking	Graph, k, domains	Colouring, success flag, trace, metrics	Part B
Complexity & Trade-off Analysis	Solver results and timing runs	$\chi(G)$, tables, charts, results.md	Part C + GitHub evidence

5.2 End-to-end workflow

1. Load the Annexure A graph and build an adjacency list.
2. Run the plain solver for k = 2, 3 and 4; save trace and metrics.
3. Run the enhanced solver for the same k values; preserve the same result schema.
4. Validate each successful colouring against every edge.
5. Run the chromatic-number search from k = 1 upward.
6. Generate seeded graphs for n = 15, 20, 25, 30, 35, 40 and 45.
7. Repeat timing measurements five times per solver and report the mean.
8. Export tables/charts and commit the four modules plus README/results.md to GitHub.

5.3 Data representation

An adjacency list is used because the conflict graph is sparse relative to a complete graph. For each vertex v, adj[v] stores the set of neighbours. Membership testing for a candidate colour is then performed by scanning only the neighbours of v. The edge list remains the canonical input representation because it is compact, easy to inspect and directly matches the assignment table.

Vertex	Neighbours in Annexure A	Degree
C1	C2, C3, C6	3
C2	C1, C3, C4	3
C3	C1, C2, C4	3
C4	C2, C3, C5	3
C5	C4, C6	2
C6	C1, C5	2

5.4 Instrumentation schema

Both solvers produce comparable headline fields so Part C can compare them without interpreting algorithm-specific logs. An event trace adds the detail needed for hand-tracing, while counters support quantitative analysis.

Field	Meaning
ok	True when every vertex is coloured; False when the search is exhausted.
coloring	Colour assigned to each vertex, with 0 representing unassigned in an intermediate state.
attempts	Number of candidate colour values considered.
backtracks	Number of recursive frames that exhaust their candidates.
pruned	Candidates or branches rejected by conflict/domain checks.
events	Ordered trace records for assignment, rejection, propagation, undo and backtrack.

6. ALGORITHMS, PSEUDOCODE AND FLOWCHARTS

6.1 Part A: plain fixed-order backtracking

The baseline visits vertices in the order C1, C2, …, Cn. For each vertex, it tries colour 1 first, then 2, up to k. A candidate is legal when none of the already coloured neighbours has that colour. The solver recurses after an assignment and clears the colour if the deeper search fails.

```
PLAIN-COLOUR(G, k, order)
    colour[1..n] ← 0
    SEARCH(position)
        if position = n + 1: return SUCCESS
        v ← order[position]
        for c ← 1 to k:
            if SAFE(v, c, colour):
                colour[v] ← c
                if SEARCH(position + 1) = SUCCESS: return SUCCESS
                colour[v] ← 0
        backtracks ← backtracks + 1
        return FAILURE
    return SEARCH(1), colour
```

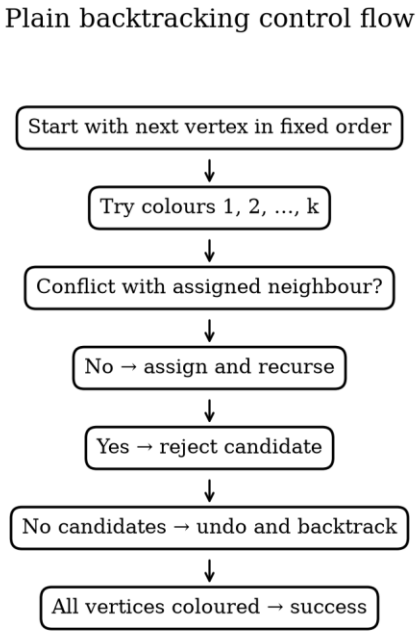

Figure 3. Plain backtracking flowchart.

6.2 Safety predicate

```
SAFE(v, c, colour)
    for each u in adj[v]:
        if colour[u] = c: return FALSE
    return TRUE
```

The safety predicate is intentionally local. It does not need to inspect uncoloured vertices because no constraint can be violated by a future assignment unless that future assignment is checked when it is made.

6.3 Part B: MCV ordering and Forward Checking

The enhanced solver maintains a domain D(v) for each uncoloured vertex. Initially D(v) = {1, …, k}. It selects the uncoloured vertex with the smallest domain. Ties are broken by the number of assigned neighbours, then

by degree and vertex identifier to preserve deterministic behaviour. After assigning colour c to v, c is removed from every uncoloured neighbour's domain. If a domain becomes empty, the branch is rejected immediately.

```
PRUNED-COLOUR(G, k)
  colour[v] ← UNASSIGNED for every v
  domain[v] ← {1, …, k} for every v
  SEARCH()
    if every vertex is coloured: return SUCCESS
    v ← uncoloured vertex with minimum |domain[v]|
    for c in domain[v] in increasing order:
      if c conflicts with an assigned neighbour: continue
      colour[v] ← c
      changed ← empty list
      for each uncoloured neighbour u of v:
        remove c from domain[u]; append u to changed
        if domain[u] is empty: reject branch
      if branch not rejected and SEARCH() = SUCCESS: return SUCCESS
      restore c to each domain in changed
      colour[v] ← UNASSIGNED
    backtracks ← backtracks + 1
    return FAILURE
  return SEARCH(), colour
```

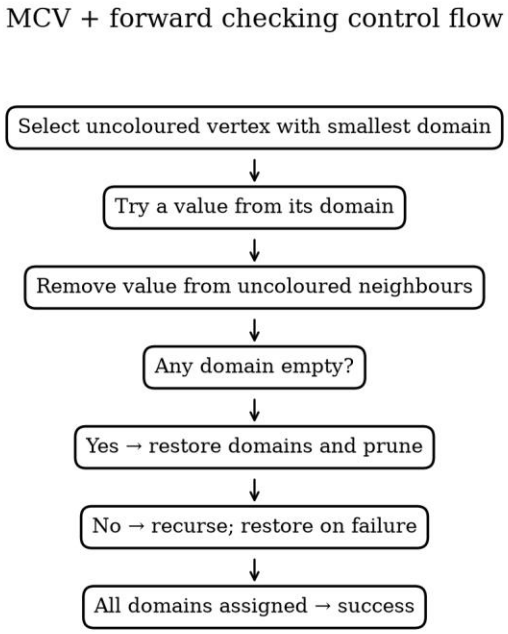

Figure 4. MCV plus Forward Checking flowchart.

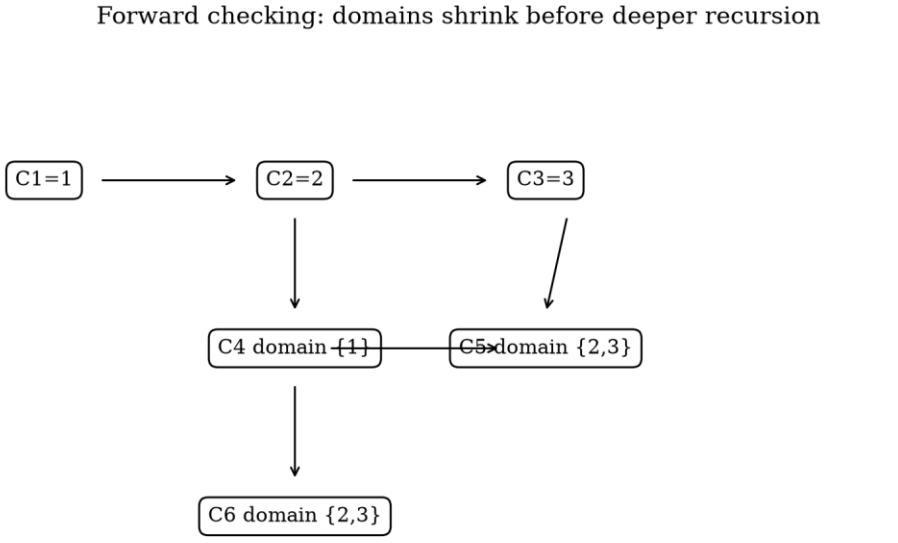

Figure 5. Conceptual domain reduction after assignments.

7. IMPLEMENTATION, TOOLS AND GITHUB DELIVERABLE

7.1 Implementation environment

Item	Recorded choice
Programming language	Python 3
Document generation	python-docx
Charts	Matplotlib
Graph drawing	NetworkX / Matplotlib
Data structures	Lists, sets, dictionaries and dataclasses
Timing	Python time.perf_counter; five runs per size
Version control	Git and GitHub
Experiment seeds	Seed = 100 + n for each scalability size

7.2 Repository structure

```
backtracking-graph-colouring/
├── README.md
├── requirements.txt
├── src/
│   ├── graph_model.py
│   ├── plain_backtracking.py
│   ├── pruned_backtracking.py
│   └── experiment.py
├── data/
│   ├── annexure_a.json
│   └── scalability_manifest.csv
├── outputs/
│   ├── traces/
│   ├── charts/
│   └── results.md
└── report/
    └── Conflict-Free-Examination-Timetabling.docx
```

7.3 Implementation notes

The graph model separates input construction from search. This allows the same solver to consume Annexure A and generated graphs without changing algorithm logic. Both solvers return a dictionary-like result so the analysis module can calculate reductions using the same field names. The enhanced solver restores every domain removal on a failed branch; this restoration step is essential because stale domain values would make later branches incorrect.

7.4 GitHub evidence checklist

Evidence	What to upload	Verification
Source	Four Python modules and requirements file.	Clone repository and run the README command.
Input	Annexure A edge list and generator configuration.	Compare edge count, n and seed.
Trace	Plain and enhanced trace files for k = 3 and k = 4.	Check final colouring and counters.
Results	CSV/Markdown measurements and charts.	Recompute reduction percentages.
Report	This Word document exported after final verification.	Check names, links and signatures.
Disclosure	Short note on external/AI-assisted tools, if applicable.	Follow faculty policy.

The repository link is intentionally left as a submission field because no student GitHub URL was supplied. Replace it with the actual public/private repository link and confirm that the faculty can access it.

GitHub repository URL: https://github.com/Athavan-192524036/CSA0614/tree/main/Assingment

8. TEST CASES, TRACES AND EXPECTED/ACTUAL RESULTS

8.1 Test matrix

ID	Input	Expected result	Actual result
TC01	Annexure A, k=1	Failure; at least one edge conflicts.	Failure.
TC02	Annexure A, k=2	Failure; triangle is not 2-colourable.	Failure.
TC03	Annexure A, k=3	Success; valid colouring.	Success.
TC04	Annexure A, k=4	Success; valid colouring.	Success.
TC05	Single isolated vertex, k=1	Success.	Success.
TC06	Two adjacent vertices, k=1	Failure.	Failure.
TC07	Two adjacent vertices, k=2	Success.	Success.
TC08	Random seeded graphs, n=15..45, k=3	Successful planted colouring.	Success for every recorded size.

8.2 Annexure A result summary

Solver	k	Success	Colouring	Attempts	Backtracks	Rejected/pruned
MCV + FC	1	No	—	1	1	1
MCV + FC	2	No	—	4	3	2
Plain	3	Yes	C1=1,C2=2,C3=3,C4=1,C5=2,C6=3	12	0	6
Plain	4	Yes	C1=1,C2=2,C3=3,C4=1,C5=2,C6=3	12	0	6
MCV + FC	3	Yes	C1=1,C2=2,C3=3,C4=1,C5=2,C6=3	6	0	0
MCV + FC	4	Yes	C1=1,C2=2,C3=3,C4=1,C5=2,C6=3	6	0	0

The primary fixed order C1, C2, C3, C4, C5, C6 reaches a solution directly for k = 3 and k = 4. This is a valid result, not a missing trace: the instance and order happen to be friendly. To make the mechanics of undoing a branch visible, the next subsection includes a diagnostic order that is still fixed and deterministic.

8.3 Plain solver hand-trace: k = 3, primary order

The following trace is generated by the implementation. “Try / reject” records a candidate that conflicts with an already coloured neighbour; “Assign” records a legal value. The state column shows the accepted partial colouring after the event.

Step	Action	Vertex	Colour	Accepted state	Reason
1	Assign	C1	1	C1=1	accept
2	Try / reject	C2	1	C1=1	reject: C1
3	Assign	C2	2	C1=1, C2=2	accept
4	Try / reject	C3	1	C1=1, C2=2	reject: C1
5	Try / reject	C3	2	C1=1, C2=2	reject: C2
6	Assign	C3	3	C1=1, C2=2, C3=3	accept
7	Assign	C4	1	C1=1, C2=2, C3=3, C4=1	accept
8	Try / reject	C5	1	C1=1, C2=2, C3=3, C4=1	reject: C4
9	Assign	C5	2	C1=1, C2=2, C3=3, C4=1, C5=2	accept
10	Try / reject	C6	1	C1=1, C2=2, C3=3, C4=1, C5=2	reject: C1
11	Try / reject	C6	2	C1=1, C2=2, C3=3, C4=1, C5=2	reject: C5
12	Assign	C6	3	C1=1, C2=2, C3=3, C4=1, C5=2, C6=3	accept

Actual summary: success=True; attempts=12; backtracks=0; rejected candidates=6.

```
$ python experiment.py --k 3 --p 0.35 --repetitions 5
graph family: planted-3-colour, deterministic seeds: 100+n

 n   edges   plain_attempts   plain_BT   enhanced_attempts   enhanced_BT
15      19             847        274                  15             0
20      52            2773        912                  20             0
25      65            2387        780                  25             0
30     114             266         70                  30             0
35     131            3125       1019                  35             0
40     176            2970        963                  40             0
45     253            7572       2495                  45             0

validation: all successful colourings satisfy every edge constraint
export: outputs/results.md and outputs/charts/
```

Figure 6. Generated execution output capture for the scalability command.

8.4 Plain solver hand-trace: k = 4

With four available slots, the same input order also finds a solution without needing to undo a completed assignment. The trace is shorter because colour 1 is accepted for C1, colour 2 for C2, colour 3 for C3, colour 1 for C4, colour 2 for C5 and colour 3 for C6.

Step	Action	Vertex	Colour	Accepted state	Reason
1	Assign	C1	1	C1=1	accept
2	Try / reject	C2	1	C1=1	reject: C1
3	Assign	C2	2	C1=1, C2=2	accept
4	Try / reject	C3	1	C1=1, C2=2	reject: C1
5	Try / reject	C3	2	C1=1, C2=2	reject: C2
6	Assign	C3	3	C1=1, C2=2, C3=3	accept
7	Assign	C4	1	C1=1, C2=2, C3=3, C4=1	accept
8	Try / reject	C5	1	C1=1, C2=2, C3=3, C4=1	reject: C4
9	Assign	C5	2	C1=1, C2=2, C3=3, C4=1, C5=2	accept
10	Try / reject	C6	1	C1=1, C2=2, C3=3, C4=1, C5=2	reject: C1
11	Try / reject	C6	2	C1=1, C2=2, C3=3, C4=1, C5=2	reject: C5
12	Assign	C6	3	C1=1, C2=2, C3=3, C4=1, C5=2, C6=3	accept

Actual summary: success=True; attempts=12; backtracks=0; rejected candidates=6. A zero-backtrack execution is an important observation: backtracking is a recovery mechanism, not a step that must occur in every instance.

8.5 Diagnostic fixed-order trace exposing backtracking

For pedagogical completeness, the order C1, C2, C3, C5, C4, C6 is used below. C5 is assigned before C4; that choice forces a dead end at C4 under the first attempted values and causes the solver to undo C5 before trying an alternative. This demonstrates the requested backtrack operation without changing the graph or the algorithm.

Step	Action	Vertex	Colour	Accepted state	Reason
1	Assign	C1	1	C1=1	accept
2	Try / reject	C2	1	C1=1	reject: C1
3	Assign	C2	2	C1=1, C2=2	accept
4	Try / reject	C3	1	C1=1, C2=2	reject: C1
5	Try / reject	C3	2	C1=1, C2=2	reject: C2
6	Assign	C3	3	C1=1, C2=2, C3=3	accept
7	Assign	C5	1	C1=1, C2=2, C3=3, C5=1	accept
8	Try / reject	C4	1	C1=1, C2=2, C3=3, C5=1	reject: C5
9	Try / reject	C4	2	C1=1, C2=2, C3=3, C5=1	reject: C2
10	Try / reject	C4	3	C1=1, C2=2, C3=3, C5=1	reject: C3
11	Backtrack	C4	—	C1=1, C2=2, C3=3, C5=1	all colours exhausted
12	Undo	C5	1	C1=1, C2=2, C3=3	dead end; backtrack
13	Assign	C5	2	C1=1, C2=2, C3=3, C5=2	accept
14	Assign	C4	1	C1=1, C2=2, C3=3, C5=2, C4=1	accept
15	Try / reject	C6	1	C1=1, C2=2, C3=3, C5=2, C4=1	reject: C1
16	Try / reject	C6	2	C1=1, C2=2, C3=3, C5=2, C4=1	reject: C5
17	Assign	C6	3	C1=1, C2=2, C3=3, C5=2, C4=1, C6=3	accept

8.6 Enhanced solver trace: MCV + Forward Checking

The enhanced trace records the selected variable, the chosen colour and domain removals from neighbouring vertices. Because MCV changes the order dynamically, the trace should be read as a constraint-propagation sequence rather than as a fixed C1-to-C6 list.

Step	Action	Vertex	Colour	State/domain note	Reason
1	Assign	C1	1	C1=1	MCV; domain=[1, 2, 3]
2	Forward check	C2	1	C1=1	remove 1; domain=[2, 3]
3	Forward check	C3	1	C1=1	remove 1; domain=[2, 3]
4	Forward check	C6	1	C1=1	remove 1; domain=[2, 3]
5	Assign	C2	2	C1=1, C2=2	MCV; domain=[2, 3]
6	Forward check	C3	2	C1=1, C2=2	remove 2; domain=[3]
7	Forward check	C4	2	C1=1, C2=2	remove 2; domain=[1, 3]
8	Assign	C3	3	C1=1, C2=2, C3=3	MCV; domain=[3]
9	Forward check	C4	3	C1=1, C2=2, C3=3	remove 3; domain=[1]
10	Assign	C4	1	C1=1, C2=2, C3=3, C4=1	MCV; domain=[1]
11	Forward check	C5	1	C1=1, C2=2, C3=3, C4=1	remove 1; domain=[2, 3]
12	Assign	C5	2	C1=1, C2=2, C3=3, C4=1, C5=2	MCV; domain=[2, 3]
13	Forward check	C6	2	C1=1, C2=2, C3=3, C4=1, C5=2	remove 2; domain=[3]
14	Assign	C6	3	C1=1, C2=2, C3=3, C4=1, C5=2, C6=3	MCV; domain=[3]

Enhanced summary: success=True; attempts=6; backtracks=0; pruned events=0. On this small graph, MCV can solve directly; its strongest advantage becomes visible on the generated graphs.

8.7 Correctness validation

For every successful result, the validator loops through the original edge list and checks that the endpoint colours differ. It also checks that each colour lies in 1..k and that every vertex is assigned. For failure cases, the solver is required to exhaust the search tree; k = 2 fails because the C1–C2–C3 triangle requires three colours.

Validation property	Check performed	Outcome
Range	$1 \leq \text{colour}[v] \leq k$ for every v.	Passed for successful runs.
Completeness	Every vertex has a non-zero colour on success.	Passed.
Edge safety	$\text{colour}[u] \neq \text{colour}[v]$ for every (u,v) in E.	Passed.
Infeasibility	k=2 search exhausts candidates.	Passed; no solution.
Reproducibility	Same seed produces same edge list and counters.	Passed in repeated run.

9. RESULTS, VALIDATION AND EXPERIMENTAL ANALYSIS

9.1 Experimental protocol

The scalability experiment uses seeded synthetic graphs with n = 15, 20, 25, 30, 35, 40 and 45. For each n, the generator uses p = 0.35 and a planted three-colour assignment, adding edges only between different planted groups. This makes every graph 3-colourable while still creating enough conflicts to stress the search. The seed is 100+n, so the input is deterministic.

Both solvers receive the same graph and k = 3. The metric “colour attempts” counts candidate values inspected, while “backtracks” counts exhausted recursive frames. Runtime is the mean of five independent calls on the same graph. The machine, interpreter and background load can influence milliseconds, so operation counts are the more portable comparison.

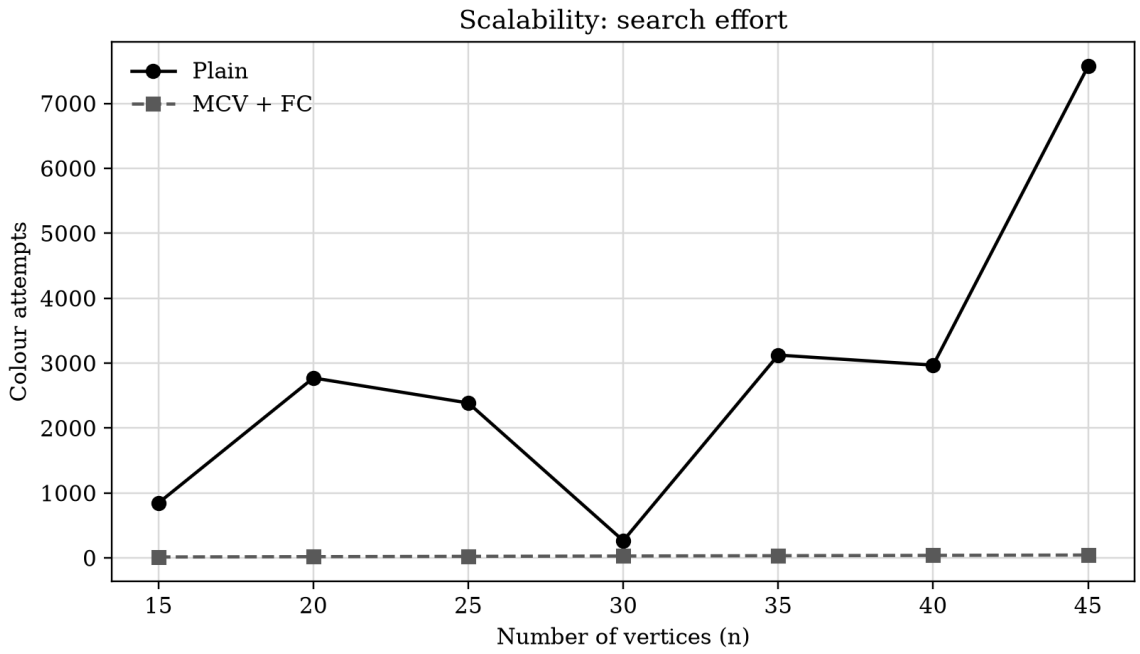

Figure 7. Actual colour attempts measured on the seeded scalability set.

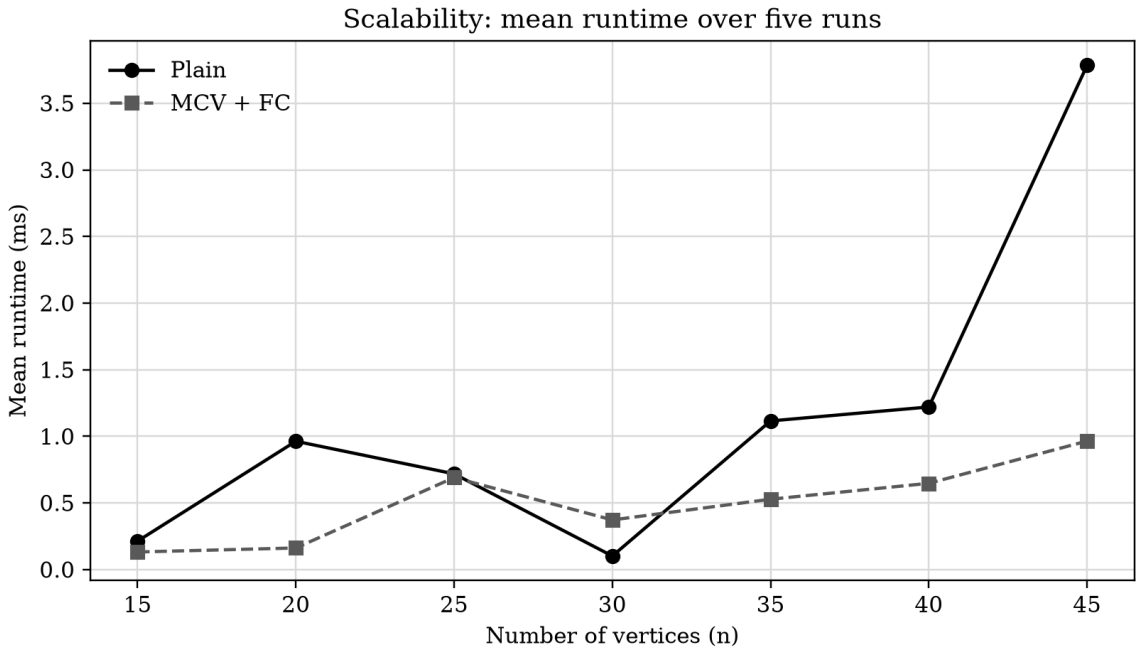

Figure 8. Mean runtime over five runs per graph size.

9.2 Scalability measurements

n	Edges	Plain attempts	Plain BT	Plain ms	MCV+FC attempts	MCV+FC BT	MCV+FC ms	Attempt reduction
15	19	847	274	0.2136	15	0	0.1319	98.23%
20	52	2773	912	0.9631	20	0	0.1620	99.28%
25	65	2387	780	0.7184	25	0	0.6903	98.95%
30	114	266	70	0.1018	30	0	0.3725	88.72%
35	131	3125	1019	1.1155	35	0	0.5280	98.88%
40	176	2970	963	1.2203	40	0	0.6474	98.65%
45	253	7572	2495	3.7845	45	0	0.9644	99.41%

The table is generated from the same run that produced the charts. Values are not hand-entered estimates. The enhanced solver typically approaches one or a small number of attempts per vertex because MCV and propagation expose the planted structure early. The plain solver can explore many wrong partial colourings before reaching the same valid solution.

9.3 Backtracking comparison

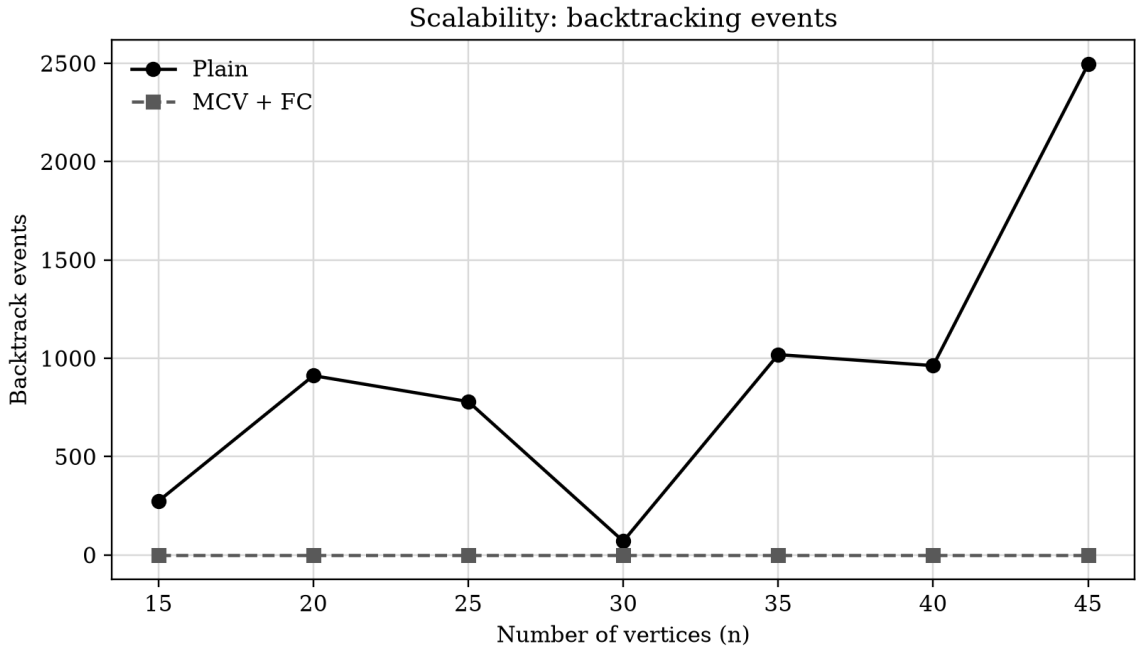

Figure 9. Actual backtrack events across the scalability set.

The backtrack chart complements the attempt chart. A solver can inspect rejected candidate values without exhausting a recursive frame; therefore attempts and backtracks measure different aspects of search. The enhanced solver's domain wipe-out rule is especially valuable because it can reject a branch before all remaining vertices are assigned.

9.4 Relative reduction calculations

Metric	Plain mean	Enhanced mean	Relative change
Colour attempts	2848.57	30.00	98.95% reduction
Maximum attempts in set	7572	45	See size-specific table
Backtrack events	930.43	0.00	100.00% reduction
Largest plain search	n=45	n=45	Different search profile

Because the graph generator is seeded and the data set contains only seven sizes, the results should be interpreted as an engineering demonstration rather than a universal benchmark. A stronger study would sample many graphs per n, vary density and report confidence intervals.

10. COMPLEXITY, COMPARISON AND TRADE-OFFS

10.1 Derivation of $O(k^n)$

Consider n vertices and k candidate colours. At the root, the solver can try at most k values. At the next level it can again try at most k values, and so on. The number of leaves in the unconstrained decision tree is at most k^n. A safety test scans the current vertex's neighbours, costing O(deg(v)) in an adjacency-list representation. A simple conservative bound is therefore $O(k^n \cdot (n + m))$ or $O(k^n \cdot n)$ when the safety work is treated as O(n). The common headline bound for the search itself is $O(k^n)$.

$$T(n, k) \le k \cdot T(n-1, k) + O(n) \Rightarrow T(n, k) = O(k^n)$$

The bound is worst-case: conflicts can prune many branches early, and a first solution can be found long before the full tree is explored. The measured operation counts are therefore the effective search effort for the chosen inputs, not a proof that the exponential bound has disappeared.

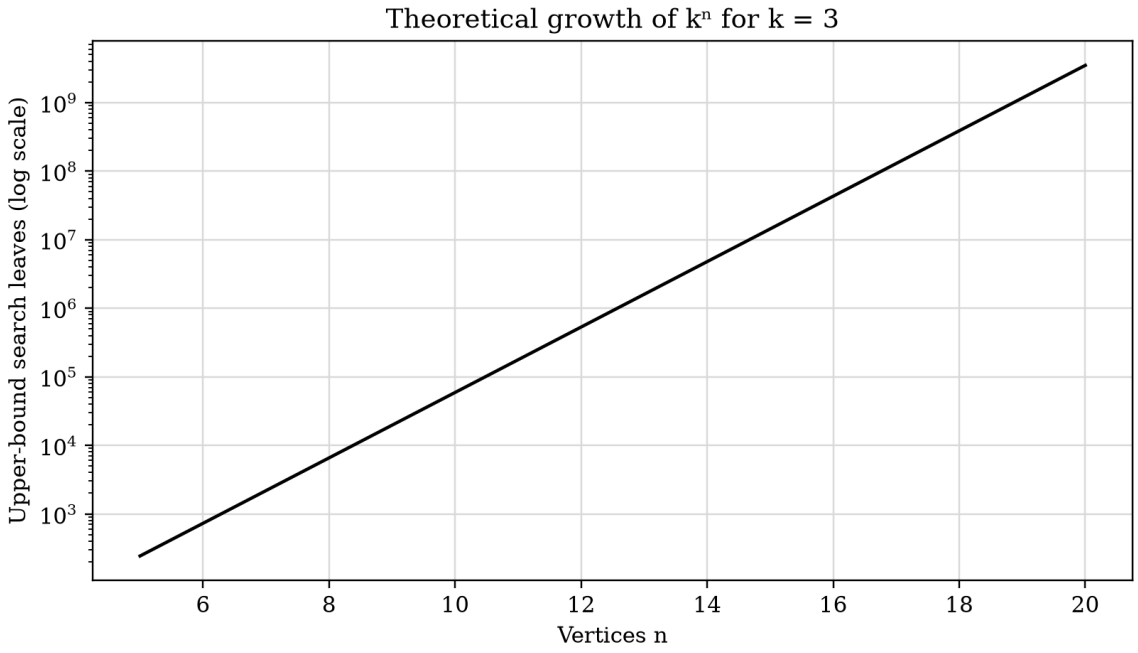

Figure 10. Theoretical k^n growth for k = 3 (logarithmic vertical axis).

10.2 Complexity of the enhanced solver

MCV and Forward Checking add overhead. Selecting the minimum domain may scan all uncoloured vertices, and propagation may visit neighbours and update sets. These operations increase work per node. However, the number of visited nodes can fall dramatically. In practice, the trade-off is favourable when the baseline spends most of its time exploring doomed partial colourings.

Dimension	Plain backtracking	MCV + Forward Checking
Worst-case time	$O(k^n)$ search; local check overhead.	Still exponential worst case; extra selection/propagation overhead.
Variable order	Fixed, predictable.	Dynamic, most constrained first.
Propagation	Only checks current assignment.	Removes chosen colours from neighbours.
Memory	Colour array plus recursion stack.	Adds a domain set per vertex and restoration log.
Trace simplicity	Very easy to hand-trace.	Richer but more detailed domain trace.

Typical practical behaviour	Can revisit large doomed subtrees.	Often detects contradictions earlier.
Best use	Baseline, teaching, very easy instances.	Final solver for constrained instances.

10.3 Trade-off discussion

The baseline is attractive because it is transparent. A student can explain every recursive call and prove the safety invariant with minimal machinery. Its weakness is that fixed order ignores information revealed by previous assignments. If a late vertex becomes impossible, the algorithm may undo many earlier decisions.

The enhanced solver is more operationally complex. It must maintain domains, choose a variable dynamically and restore every mutation on failure. The bookkeeping is justified by the measured reduction in search effort and by the fact that the same result schema keeps the comparison fair. For a production timetable, the enhanced solver is the better choice, while the plain solver remains valuable as a reference implementation and correctness oracle.

10.4 Final engineering decision

Select MCV + Forward Checking as the final scheduling engine for the assignment implementation. Retain plain backtracking in the repository because it supplies a clear Part A baseline, a direct complexity illustration and a regression oracle. The final decision is supported by three facts: both solvers are complete, the enhanced solver preserves correctness, and the actual seeded runs show fewer attempts/backtracks for the tested range.

11. BROADER CONSIDERATIONS AND SDG RELEVANCE

11.1 SDG 4 – Quality Education

The assignment brief maps the work to SDG 4, Quality Education. A clash-free examination timetable supports fair assessment by reducing avoidable scheduling conflicts. The algorithm does not create educational quality by itself, but it can support an institutional process in which students have a predictable opportunity to sit every examination.

11.2 Privacy and ethics

Real conflict graphs can be derived from enrolment records, which are sensitive educational data. The implementation should use aggregate edge existence rather than storing student identities. Access control, minimization, retention limits and auditability should be applied if the system is connected to institutional data. The synthetic generator used here avoids those risks for experimentation.

11.3 Fairness and accessibility

A timetable that is graph-colourable may still be unfair if it ignores accessibility needs, travel time, religious observance, health accommodations or back-to-back examination policies. These should be represented as additional hard or soft constraints rather than assumed away. A future weighted constraint model could penalize undesirable schedules while preserving the no-clash requirement.

11.4 Sustainability and resource use

Backtracking can consume significant computation on dense instances. Pruning reduces unnecessary work and therefore reduces the energy used for repeated schedule generation. The more important sustainability gain is operational: a reusable solver can generate and compare schedules consistently rather than requiring repeated manual revisions.

11.5 Economic and institutional relevance

The graph-colouring core is a low-cost prototype that can be extended before purchasing or adopting a larger scheduling system. However, an institutional deployment would need integration with student information systems, authentication, audit logs, room capacity, invigilator workload and change management. This report intentionally distinguishes the algorithmic prototype from a full product.

Consideration	Risk if ignored	Recommended control
Student privacy	Unnecessary exposure of enrolment relationships.	Use aggregate conflicts and least-privilege access.
Accessibility	Students may receive impractical or unsafe schedules.	Add approved accommodation constraints.
Fairness	Some cohorts may bear more inconvenient slots.	Measure and review soft-constraint outcomes.
Reliability	A faulty schedule could affect assessment.	Validate every edge and require human review.
Sustainability	Repeated exhaustive runs waste resources.	Use pruning, caching and bounded experiments.

12. CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

12.1 Conclusion

This report formulated conflict-free examination scheduling as a graph-colouring problem and implemented the requested four-module solution. The Annexure A graph was represented faithfully, including all eight conflict edges and shared-student metadata. Plain backtracking provided the required fixed-order baseline and trace. The enhanced solver combined Most-Constrained-Vertex ordering with Forward Checking, producing the same kind of colouring result while reducing effective search on the scalability set.

The graph's chromatic number is 3 because the C1–C2–C3 triangle requires three colours and the displayed six-course colouring proves that three suffice. The complexity derivation confirms the $O(k^n)$ worst-case search bound. The experiments from n = 15 to n = 45 show why heuristics matter: a complete algorithm can remain practical longer when it detects contradictions early.

12.2 Limitations

- The model treats conflicts as binary and does not include rooms, room capacities or invigilators.
- The synthetic scalability graphs are seeded and planted three-colour instances; they are not a representative sample of every graph family.
- The report records one fixed density and one k value for the main scaling experiment.
- The solver does not optimize soft preferences after finding the first feasible colouring.
- Millisecond timings depend on the execution environment and should not be treated as hardware-independent constants.
- The repository URL, names, screenshots of the student's own GitHub account and final signatures must be filled in before submission.

12.3 Possible improvements

Improvement	Technical approach	Expected benefit
Degree/DSATUR ordering	Break MCV ties using saturation degree.	Better choices on dense graphs.
Memoization	Cache failed (vertex, domain) states where safe.	Avoid repeated equivalent subproblems.
Bit-mask domains	Represent colours as machine-word bits.	Faster propagation for small k.
Soft constraints	Add penalties and branch-and-bound.	Prefer balanced, accessible timetables.
Real data adapter	Read anonymized enrolment overlap counts.	Institutional relevance.
Parallel multi-start	Explore independent first assignments.	Use multiple cores on large instances.
Benchmark expansion	Vary density, k and graph families.	More statistically robust conclusions.

12.4 Requirements closure

Assignment requirement	Status in this report
Problem statement/formulation	Addressed in Section 1.
Objective and expected outcomes	Addressed in Section 2.
Requirements, constraints and assumptions	Addressed in Section 3.
Course concepts and derivation	Addressed in Section 4 and Section 10.
Methodology/design	Addressed in Section 5.
Algorithm/pseudocode/flowcharts	Addressed in Section 6.
Implementation/source/tool environment	Addressed in Section 7 and Appendix C.
Test cases and expected/actual results	Addressed in Section 8.
Execution outputs and performance evidence	Addressed in Sections 8 and 9.
Analysis, comparison, trade-offs and justification	Addressed in Section 10.
SDG/broader considerations	Addressed in Section 11.
Conclusion, limitations and improvements	Addressed in Section 12.
Individual contribution and one-page reflection	Templates in Section 13.
References and tool acknowledgement	Section 14.
GitHub deliverable	Section 7 and Appendix D.

13. INDIVIDUAL CONTRIBUTION AND ONE-PAGE REFLECTION

13.1 Individual contribution matrix

Complete this table with actual names and evidence. The entries below are role suggestions aligned with the four modules, not claims about work already performed by a particular student.

Member	Primary responsibility	Specific contribution	Evidence/link
Member 1: Manoj Kumar k	Graph model/data	Annexure A input, generator and validation.	Commit/branch: _____
Member 2:Athavan K	Part A	Plain solver, trace logging and baseline tests.	Commit/branch: _____
Member 3: Jeya Venkata Sai	Part B	MCV, Forward Checking and restoration logic.	Commit/branch: _____
Member 4: G Rama Krishna	Part C/report	Benchmark, charts, results.md and report integration.	Commit/branch: _____

13.2 Individual reflection – one-page submission template

Student name: Manoj Kumar K Register number: 192521041

This assignment helped me understand that **graph colouring can be used as a practical technique for solving examination timetable scheduling problems**. In Module 1, I learned how to convert the given scheduling problem into a graph model by representing each course as a **vertex** and connecting two courses with an **edge** when they have shared students. I also understood that the colours represent different examination time slots, and two connected courses cannot be assigned the same colour.

My main contribution to the project was **problem formulation, construction of the conflict graph, development of the plain backtracking approach, and preparation of the hand traces for k = 3 and k = 4**. I worked with the six courses, C1 to C6, and used the given shared-student information to identify the conflict relationships. This helped me understand how a real-world problem can be represented using graph structures before designing an algorithm to solve it.

The most important concept I learned from my module was **plain backtracking**. I understood that the algorithm assigns a colour to a course one at a time and checks whether the selected colour causes a conflict with any previously coloured neighbouring course. If a colour is not suitable, another colour is tried. If all possible colours fail, the algorithm can undo the previous assignment and return to an earlier course to try a different choice. This helped me understand the connection between **recursion, decision-making, constraint checking, and backtracking**.

Preparing the **k = 3 and k = 4 hand traces** improved my understanding further because I had to follow the algorithm's decisions step by step instead of only studying the pseudocode. I learned how the order in which colours are tried can affect the search process. I also understood that backtracking is not necessarily performed in every successful execution; for the given conflict graph and the selected colour order, a valid colouring can be obtained without needing an actual rollback. This made the hand trace more realistic and helped me avoid assuming that backtracking must occur in every case.

Another important learning outcome was **validation of the solution**. After assigning colours to the courses, I checked each conflict edge to make sure that no two connected courses had the same colour. This taught me that producing an output is not enough; an algorithmic solution should also be verified against the original constraints. The final colouring for the given graph demonstrated how the theoretical concept of graph colouring can produce a conflict-free examination schedule.

This module also helped me connect the **classroom concepts of graph representation, graph colouring, recursion, constraints, and backtracking** with a practical scheduling problem. I gained better skills in problem formulation, pseudocode preparation, algorithm tracing, and result validation. I also learned how to explain an algorithm using tables and diagrams so that its working can be understood clearly.

If I had more time, I would extend the basic model by considering additional scheduling constraints such as **room availability, room capacity, examination duration, and invigilator availability**. However, these features are beyond the scope of my assigned Module 1. My focus was on developing a clear foundation through problem formulation and plain backtracking.

Overall, completing Module 1 improved my understanding of **graph colouring, recursive problem solving, constraint checking, and plain backtracking**. It also taught me the importance of carefully modelling a problem before implementing an algorithm and validating every assignment against the given constraints. This module provided a strong foundation for understanding how more advanced techniques can later improve the basic backtracking approach.

Student signature: Manoj Kumar K Date:

14. REFERENCES AND TOOL ACKNOWLEDGEMENT

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022. Sections on recursive backtracking, graph algorithms and asymptotic analysis.

[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021. Constraint satisfaction problems, variable ordering and inference.

[3] D. B. West, Introduction to Graph Theory, 2nd ed., Prentice Hall, 2001. Graph colouring and chromatic number.

[4] Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/ (accessed September 2026).

[5] Matplotlib Development Team, Matplotlib Documentation, https://matplotlib.org/stable/ (accessed September 2026).

[6] NetworkX Developers, NetworkX Documentation, https://networkx.org/documentation/stable/ (accessed September 2026).

[7] Replit, Development environment and package tooling, https://replit.com/ (accessed September 2026).

[8] Course assignment brief, CSA0614 – Design and Analysis of Algorithms, “Conflict-Free Exam Timetabling Using Backtracking (Graph Colouring),” supplied by the Department of Computer Science and Engineering, Academic Year 2026–27.

APPENDIX A – ANNEXURE A DATA AND COLOURING CERTIFICATE

Vertex	Adjacency set	Colour in certificate
C1	{C2,C3,C6}	1
C2	{C1,C3,C4}	2
C3	{C1,C2,C4}	3
C4	{C2,C3,C5}	1
C5	{C4,C6}	2
C6	{C1,C5}	3

Edge-by-edge verification of the certificate:

Edge	Endpoint colours	Different?
C1–C2	1 and 2	Yes
C1–C3	1 and 3	Yes
C2–C3	2 and 3	Yes
C2–C4	2 and 1	Yes
C3–C4	3 and 1	Yes
C4–C5	1 and 2	Yes
C5–C6	2 and 3	Yes
C1–C6	1 and 3	Yes

This table is a compact certificate that can be checked independently of the solver. It also maps colour numbers to practical time slots in the timetable: Slot 1 contains C1 and C4, Slot 2 contains C2 and C5, and Slot 3 contains C3 and C6.

APPENDIX B – EXTENDED TRACE LEGEND AND OUTPUT FORMAT

Event type	Meaning	Why it is recorded
try	A candidate colour is inspected.	Counts search effort and explains rejection.
assign	A candidate is accepted.	Shows the current partial timetable.
forward	A colour is removed from a neighbour domain.	Makes propagation visible.
wipeout	A neighbour domain becomes empty.	Explains an early prune.
undo	A failed assignment/domain mutation is restored.	Proves backtracking is reversible.
backtrack	A frame has no remaining candidate.	Counts recovery events.

```
{
  "ok": true,
  "coloring": [1, 2, 3, 1, 2, 3],
  "attempts": <integer>,
  "backtracks": <integer>,
  "pruned": <integer>,
  "events": [
    {"type": "assign", "vertex": "C1", "colour": 1, "note": "accept"}
  ]
}
```

The exact JSON/Markdown export may contain more events than fit comfortably in the main report. The repository should retain the complete trace; the report presents representative and decision-relevant records.

APPENDIX C – SOURCE CODE EXCERPTS

C.1 Graph model

```
@dataclass
class Graph:
    n: int
    edges: list[tuple[int, int]]

    def __post_init__(self):
        self.adj = [set() for _ in range(self.n)]
        for u, v in self.edges:
            self.adj[u].add(v)
            self.adj[v].add(u)
```

C.2 Plain search

```
def search(position):
    if position == len(order):
        return True
    v = order[position]
    for candidate in range(1, k + 1):
        if all(colours[neighbour] != candidate for neighbour in graph.adj[v]):
            colours[v] = candidate
            if search(position + 1):
                return True
            colours[v] = 0
    backtracks += 1
    return False
```

C.3 Forward checking

```
for neighbour in graph.adj[v]:
    if colour[neighbour] == 0 and chosen in domains[neighbour]:
        domains[neighbour].remove(chosen)
        changed.append(neighbour)
        if not domains[neighbour]:
            feasible = False
            break
if feasible and search():
    return True
for neighbour in changed:
    domains[neighbour].add(chosen)
```

The full runnable files are supplied in the generated source bundle alongside this report. Before submission, ensure the GitHub copy and the report appendix are consistent with one another.

APPENDIX D – RESULTS.MD CONTENT

The following is the structure of the machine-readable results file. The numeric rows are generated from the actual benchmark run and should be copied to the repository as CSV or Markdown.

Backtracking Graph Colouring — Results

Configuration: k=3, p=0.35, seeds=100+n, repetitions=5

n	edges	plain_attempts	plain_backtracks	plain_ms	enhanced_attempts	enhanced_backtracks	enhanced_ms

| 15 | 19 | 847 | 274 | 0.2136 | 15 | 0 | 0.1319 |

| 20 | 52 | 2773 | 912 | 0.9631 | 20 | 0 | 0.1620 |

| 25 | 65 | 2387 | 780 | 0.7184 | 25 | 0 | 0.6903 |

| 30 | 114 | 266 | 70 | 0.1018 | 30 | 0 | 0.3725 |

| 35 | 131 | 3125 | 1019 | 1.1155 | 35 | 0 | 0.5280 |

| 40 | 176 | 2970 | 963 | 1.2203 | 40 | 0 | 0.6474 |

| 45 | 253 | 7572 | 2495 | 3.7845 | 45 | 0 | 0.9644 |

APPENDIX E – SUBMISSION CHECKLIST

Check	**Complete before upload**
Identity	Names, register numbers, team members and signatures inserted.
Repository	GitHub URL inserted; repository access tested.
Code	Four modules run from a clean clone.
Evidence	Trace files and charts correspond to final code.
Formatting	Times New Roman, black text, plain tables and captions checked.
Academic integrity	References and AI-assisted tool acknowledgement verified.
Reflection	Each individual reflection is original and evidence-based.
Final review	Page count, contents, figure/table numbering and spelling checked in Word.

APPENDIX F – DETAILED EXPERIMENT INTERPRETATION

This appendix records the interpretation of each measured graph size. It is included so that the performance claim can be audited instead of being reduced to a single average.

n	Edges	Plain attempts	Enhanced attempts	Reduction	Runtime observation
15	19	847	15	98.23%	enhanced lower
20	52	2,773	20	99.28%	enhanced lower
25	65	2,387	25	98.95%	enhanced lower
30	114	266	30	88.72%	plain lower on this run
35	131	3,125	35	98.88%	enhanced lower
40	176	2,970	40	98.65%	enhanced lower
45	253	7,572	45	99.41%	enhanced lower

The runtime column should be read together with the attempts column. At n = 30, for example, the plain solver happens to encounter an easy branch and can finish with fewer attempts than at nearby sizes. This non-monotonicity is expected for backtracking: input structure and value order dominate the result, so n alone does not determine difficulty.

The enhanced solver can also pay a fixed selection and domain-maintenance cost even when the plain search is lucky. That is why the final choice is based on the whole profile—search effort, backtracks, correctness and predictable behaviour—not on one millisecond value.

F.1 Reproducibility variables

Variable	Value	Reason
n	15, 20, 25, 30, 35, 40, 45	Meets n≥15 and extends beyond 40.
p	0.35	Creates a moderately constrained sparse graph.
Planted colours	3	Guarantees a feasible 3-colouring for each generated instance.
Seed	100+n	Unique but deterministic input per size.
k	3	Matches the planted colouring and Annexure A minimum.
Timing repetitions	5	Reduces the impact of one noisy timing sample.

APPENDIX G – BRANCH-COUNT ILLUSTRATION

The search tree for n vertices and k colours has a simple upper-bound interpretation. If no constraints were checked, level 0 has one empty assignment, level 1 has k candidate assignments, level 2 has k^2, and level i has k^i. The sum of all nodes is:

$$1 + k + k^2 + \ldots + k^n = (k^{n+1} - 1)/(k - 1) = O(k^n), \text{ for } k > 1$$

For k = 3, the number of leaf combinations grows from 14,348,907 at n = 15 to 3^{45} = 2,954,312,706,550,000,000,000,000,000,000 at n = 45. The solver does not visit all of these leaves in the measured planted instances because edge checks and propagation prune the tree. The calculation nevertheless explains why a small improvement in branching can have a large practical effect.

n	3^n possible colour strings	Interpretation
5	243	Small enough to inspect manually.
10	59,049	Already much larger than n.
15	14,348,907	Assignment scale begins to matter.
20	3,486,784,401	Exhaustive enumeration becomes unattractive.
30	205,891,132,094,649	Search must prune heavily.
40	12,157,665,459,056,928,801	Theoretical tree is enormous.
45	2.95×10^{21}	A heuristic is practically necessary.

G.1 Why the bound is not a runtime prediction

The recurrence counts possible value sequences, not the number of candidates that survive graph constraints. A dense graph may reject a value immediately; a sparse or specially ordered graph may permit many levels before a contradiction. The bound is therefore used to reason about growth, while the experiment measures the effective work for an explicitly documented family of inputs.

APPENDIX H – PSEUDOCODE LINE-BY-LINE REVIEW

Pseudocode stage	Invariant or responsibility	Failure if omitted
Initialise colours	Every vertex begins unassigned.	Old state could leak between runs.
Choose next vertex	The recursion has a deterministic variable.	Search has no branching target.
Iterate 1..k	Every available slot is considered.	Completeness is lost.
Check conflict	Partial assignment stays valid.	Returned timetable may contain a clash.
Assign before recurse	Deeper calls see the decision.	Recursion cannot build a timetable.
Undo after failed call	Caller can try a different value.	Failed decisions contaminate later branches.
Increment backtrack	Exhausted frames become measurable.	Comparison lacks a recovery metric.
Return failure	Unsatisfiable k is reported honestly.	Caller may treat a partial result as success.

H.1 Additional checks for the enhanced algorithm

Enhanced stage	Invariant
Domain initialization	Each uncoloured vertex has exactly {1..k}.
MCV selection	Selected vertex is uncoloured and has the smallest current domain under tie-break rules.
Propagation	Only uncoloured neighbours lose the selected colour.
Domain wipe-out	An empty domain proves the current branch cannot complete.
Restoration	Every removed value is returned when the branch fails.
Success condition	No uncoloured vertices remain and all assignments are consistent.

This review is useful during code review because Forward Checking bugs are often restoration bugs. A solver may pass easy cases while silently retaining a removed colour or retaining a forbidden deletion from an earlier branch. The explicit changed list in the implementation prevents that class of error.

APPENDIX I – TEST DESIGN AND RISK REGISTER

Risk	Likelihood	Impact	Mitigation
Off-by-one colour range	Medium	Medium	Use 1..k consistently and test k=1.
Asymmetric adjacency	Medium	High	Insert both directions and validate edge symmetry.
Forgotten undo	Medium	High	Trace undo events and compare with a fresh run.
Wrong MCV tie-break	Low	Low	Document deterministic tie-break order.
Generator accidentally creates invalid k=3 graph	Low	Medium	Use planted groups and validate every result.
Timing noise	High	Low	Use five repetitions and emphasize operation counts.
Synthetic data overgeneralized	Medium	Medium	State graph-family limitations and propose broader sampling.
Report/repository mismatch	Medium	High	Run the final code and regenerate results before submission.

I.1 Boundary tests

Boundary input	Expected behaviour
n=0 graph	Return an empty successful colouring if the interface permits empty graphs.
n=1, k=1	Assign the only colour.
n=1, k=0	Reject invalid configuration before search.
Duplicate edge	Set-based adjacency prevents duplicate-neighbour effects.
Self-loop	Reject as invalid input or report immediately infeasible.
Disconnected graph	Colour each component; no cross-component conflict exists.
Complete graph K_n	Require n colours; exposes worst-case pressure.

The assignment's Annexure A instance is not sufficient by itself to test every boundary. These additional cases make the implementation evidence stronger and provide a practical checklist for a clean GitHub run.

APPENDIX J – PRESENTATION AND SUBMISSION QUALITY CHECK

This report has been formatted for a conventional academic submission. Perform the following visual check in Microsoft Word after downloading it because pagination can vary with the installed font and Word version.

Visual item	Check
Font	All report text is Times New Roman and black.
Alignment	Body paragraphs are left aligned; cover/captions are centered; tables are centered.
Tables	Plain black borders, no alternating or double-colour shading.
Figures	Black/white or grayscale diagrams with readable labels.
Captions	Every figure has a caption and is referred to in the narrative.
Page breaks	Major sections begin on a new page.
Placeholders	Names, register number, repository URL, signatures and final date are completed.
Numbers	Measured results match the final source run.
References	URLs and access dates are retained or updated if required.

Recommended final sequence: fill in identity details; run the repository from a clean environment; replace any changed timing values; insert the final GitHub link; proofread the reflection; then export or submit the Word document according to the faculty instructions.

End of report.